bike

February 23, 2022

1 Bike Sharing Demand

Bike sharing systems are a means of renting bicycles where the process of obtaining membership, rental, and bike return is automated via a network of kiosk locations throughout a city. Using these systems, people are able rent a bike from a one location and return it to a different place on an as-needed basis. Currently, there are over 500 bike-sharing programs around the world.

The data generated by these systems makes them attractive for researchers because the duration of travel, departure location, arrival location, and time elapsed is explicitly recorded. Bike sharing systems therefore function as a sensor network, which can be used for studying mobility in a city. In this competition, participants are asked to combine historical usage patterns with weather data in order to forecast bike rental demand in the Capital Bikeshare program in Washington, D.C.

2 Load Dataset

[ ]:
```
# import pandas
import pandas as pd
# import data
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
sample_submission = pd.read_csv('sampleSubmission.csv')
```

You are provided hourly rental data spanning two years. For this competition, the training set is comprised of the first 19 days of each month, while the test set is the 20th to the end of the month. You must predict the total count of bikes rented during each hour covered by the test set, using only information available prior to the rental period.

Data Fields: * datetime - hourly date + timestamp
* season * 1 = spring * 2 = summer * 3 = fall * 4 = winter * holiday - whether the day is considered a holiday * workingday - whether the day is neither a weekend nor holiday * weather * 1: Clear, Few clouds, Partly cloudy, Partly cloudy * 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist * 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds * 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog * temp - temperature in Celsius * atemp - “feels like” temperature in Celsius * humidity - relative humidity * windspeed - wind speed * casual - number of non-registered user rentals initiated * registered - number of registered user rentals initiated * count - number of total rentals

3 Summarize Data

[ ]:
```
# head
train.head()
```

[ ]:
```
             datetime  season  holiday  workingday  weather  temp   atemp  \
0  2011-01-01 00:00:00       1        0           0        1  9.84  14.395
1  2011-01-01 01:00:00       1        0           0        1  9.02  13.635
2  2011-01-01 02:00:00       1        0           0        1  9.02  13.635
3  2011-01-01 03:00:00       1        0           0        1  9.84  14.395
4  2011-01-01 04:00:00       1        0           0        1  9.84  14.395

   humidity  windspeed  casual  registered  count
0        81        0.0       3          13     16
1        80        0.0       8          32     40
2        80        0.0       5          27     32
3        75        0.0       3          10     13
4        75        0.0       0           1      1
```

[ ]:
```
# info
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   datetime    10886 non-null  object
 1   season      10886 non-null  int64
 2   holiday     10886 non-null  int64
 3   workingday  10886 non-null  int64
 4   weather     10886 non-null  int64
 5   temp        10886 non-null  float64
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64
 10  registered  10886 non-null  int64
 11  count       10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

3.1 Descriptive Statistics

3.1.1 Categorical Features

[ ]:
```
# get categorical features
categorical_features = list(train.select_dtypes(include=['object', 'category']).
 ↪columns)
# statistics
train[categorical_features].describe()
```

[ ]:
```
                   datetime
count                 10886
unique                10886
top     2011-10-19 04:00:00
freq                      1
```

3.1.2 Numeric Features

[ ]:
```
# get numeric features
numeric_features = list(train.select_dtypes(include=['int', 'float']).columns)
# summary statistics
display(train[numeric_features].describe())
```

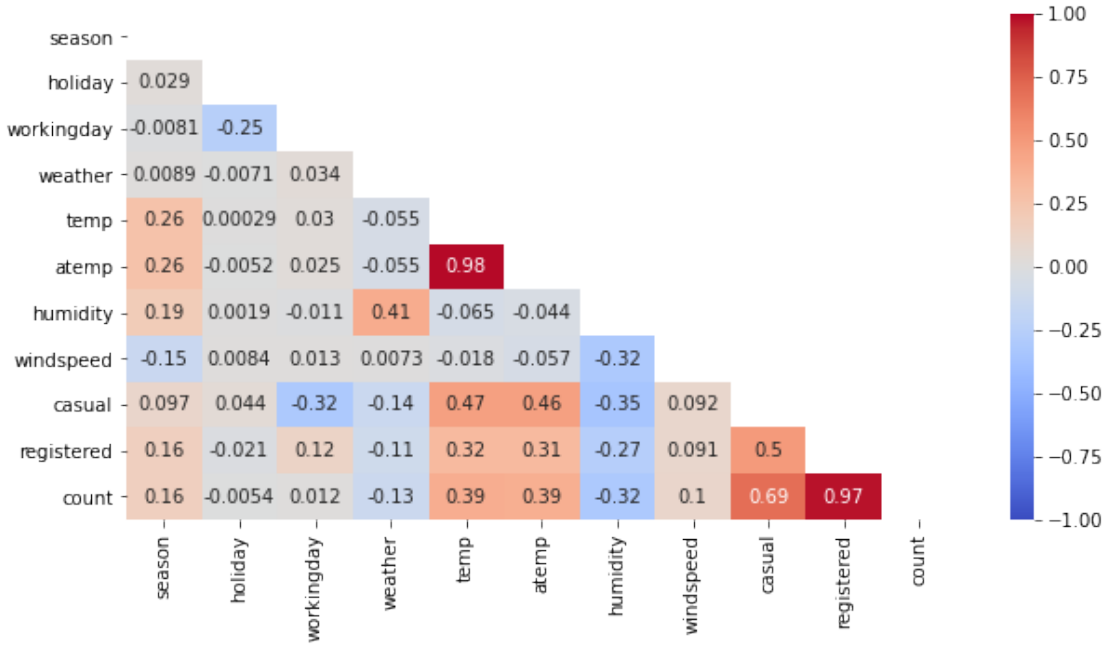
```
             season       holiday    workingday       weather          temp  \
count  10886.000000  10886.000000  10886.000000  10886.000000  10886.00000
mean       2.506614      0.028569      0.680875      1.418427     20.23086
std        1.116174      0.166599      0.466159      0.633839      7.79159
min        1.000000      0.000000      0.000000      1.000000      0.82000
25%        2.000000      0.000000      0.000000      1.000000     13.94000
50%        3.000000      0.000000      1.000000      1.000000     20.50000
75%        4.000000      0.000000      1.000000      2.000000     26.24000
max        4.000000      1.000000      1.000000      4.000000     41.00000

              atemp      humidity     windspeed        casual    registered  \
count  10886.000000  10886.000000  10886.000000  10886.000000  10886.000000
mean      23.655084     61.886460     12.799395     36.021955    155.552177
std        8.474601     19.245033      8.164537     49.960477    151.039033
min        0.760000      0.000000      0.000000      0.000000      0.000000
25%       16.665000     47.000000      7.001500      4.000000     36.000000
50%       24.240000     62.000000     12.998000     17.000000    118.000000
75%       31.060000     77.000000     16.997900     49.000000    222.000000
max       45.455000    100.000000     56.996900    367.000000    886.000000

              count
count  10886.000000
mean     191.574132
std      181.144454
min        1.000000
25%       42.000000
50%      145.000000
```

```
75%       284.000000
max       977.000000
```

[ ]:
```
# correlation heatmap
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
matrix = np.triu(train.corr(method='pearson'))
plt.figure(figsize=(10,5))
sns.heatmap(train.corr(method='pearson'),annot=True, vmin=-1, vmax=1, center=␣
 ↪0, mask=matrix, cmap='coolwarm', fmt='.2g')
plt.show()
# correlation
train.corr()
```



[ ]:
```
                season   holiday  workingday   weather      temp     atemp  \
season        1.000000  0.029368   -0.008126  0.008879  0.258689  0.264744
holiday       0.029368  1.000000   -0.250491 -0.007074  0.000295 -0.005215
workingday   -0.008126 -0.250491    1.000000  0.033772  0.029966  0.024660
weather       0.008879 -0.007074    0.033772  1.000000 -0.055035 -0.055376
temp          0.258689  0.000295    0.029966 -0.055035  1.000000  0.984948
atemp         0.264744 -0.005215    0.024660 -0.055376  0.984948  1.000000
humidity      0.190610  0.001929   -0.010880  0.406244 -0.064949 -0.043536
windspeed    -0.147121  0.008409    0.013373  0.007261 -0.017852 -0.057473
casual        0.096758  0.043799   -0.319111 -0.135918  0.467097  0.462067
registered    0.164011 -0.020956    0.119460 -0.109340  0.318571  0.314635
```

```
count       0.163439 -0.005393    0.011594 -0.128655  0.394454  0.389784

            humidity  windspeed    casual  registered     count
season      0.190610  -0.147121  0.096758    0.164011  0.163439
holiday     0.001929   0.008409  0.043799   -0.020956 -0.005393
workingday -0.010880   0.013373 -0.319111    0.119460  0.011594
weather     0.406244   0.007261 -0.135918   -0.109340 -0.128655
temp       -0.064949  -0.017852  0.467097    0.318571  0.394454
atemp      -0.043536  -0.057473  0.462067    0.314635  0.389784
humidity    1.000000  -0.318607 -0.348187   -0.265458 -0.317371
windspeed  -0.318607   1.000000  0.092276    0.091052  0.101369
casual     -0.348187   0.092276  1.000000    0.497250  0.690414
registered -0.265458   0.091052  0.497250    1.000000  0.970948
count      -0.317371   0.101369  0.690414    0.970948  1.000000
```

4 Data Cleaning

4.1 Converting to categorical features

[ ]:
```
# rename season levels
train['season'] = train['season'].replace({1: 'Spring', 2: 'Summer', 3: 'Fall',␣
 ↪4: 'Winter'})
test['season'] = test['season'].replace({1: 'Spring', 2: 'Summer', 3: 'Fall', 4:
 ↪ 'Winter'})

# rename workingday levels
train['workingday'] = train['workingday'].replace({0: 'No', 1: 'Yes'})
test['workingday'] = test['workingday'].replace({0: 'No', 1: 'Yes'})

# rename holiday levels
train['holiday'] = train['holiday'].replace({0: 'No', 1: 'Yes'})
test['holiday'] = test['holiday'].replace({0: 'No', 1: 'Yes'})

# rename weather
train['weather'] = train['weather'].replace({1: 'Clear', 2:'Cloudy', 3:'Light␣
 ↪Precipitation', 4:'Heavy Precipitation'})
test['weather'] = test['weather'].replace({1: 'Clear', 2:'Cloudy', 3:'Light␣
 ↪Precipitation', 4:'Heavy Precipitation'})
```

4.2 Temp vs Atemp

Both features measure the weather. Temp has a higher correlation with the target, variable count.

[ ]:
```
# correlation
display(train.corr())
```

```
                temp     atemp  humidity  windspeed    casual  registered  \
temp        1.000000  0.984948 -0.064949  -0.017852  0.467097    0.318571
```

```
atemp       0.984948  1.000000 -0.043536  -0.057473  0.462067    0.314635
humidity   -0.064949 -0.043536  1.000000  -0.318607 -0.348187   -0.265458
windspeed  -0.017852 -0.057473 -0.318607   1.000000  0.092276    0.091052
casual      0.467097  0.462067 -0.348187   0.092276  1.000000    0.497250
registered  0.318571  0.314635 -0.265458   0.091052  0.497250    1.000000
count       0.394454  0.389784 -0.317371   0.101369  0.690414    0.970948

                count
temp         0.394454
atemp        0.389784
humidity    -0.317371
windspeed    0.101369
casual       0.690414
registered   0.970948
count        1.000000
```

[ ]:
```
# drop atemp
train = train.drop('atemp', axis='columns')
test = test.drop('atemp', axis='columns')
```

4.3 Dropping Features

The features, 'casual' & 'registered', are not in the test set. It provides no predictive power.

[ ]:
```
# drop features
train = train.drop(['registered', 'casual'], axis='columns')
```

4.4 Dates

[ ]:
```
# Date Fields
train['datetime'] = pd.to_datetime(train['datetime'])
train['dt_year'] = train['datetime'].dt.year.astype('category')
train['dt_month'] = train['datetime'].dt.month_name().astype('category')
train['dt_hour'] = train['datetime'].dt.hour.astype('category')
train['dt_day'] = train['datetime'].dt.day_name().astype('category')
train = train.drop('datetime', axis='columns')

test['datetime'] = pd.to_datetime(test['datetime'])
test['dt_year'] = test['datetime'].dt.year.astype('category')
test['dt_month'] = test['datetime'].dt.month_name().astype('category')
test['dt_hour'] = test['datetime'].dt.hour.astype('category')
test['dt_day'] = test['datetime'].dt.day_name().astype('category')
test = test.drop('datetime', axis='columns')
train.head()
```

[ ]:
```
   season holiday workingday weather  temp  humidity  windspeed  count  \
0  Spring      No         No   Clear  9.84        81        0.0     16
1  Spring      No         No   Clear  9.02        80        0.0     40
```

```
2  Spring      No         No   Clear  9.02        80        0.0     32
3  Spring      No         No   Clear  9.84        75        0.0     13
4  Spring      No         No   Clear  9.84        75        0.0      1

  dt_year dt_month dt_hour    dt_day
0    2011  January       0  Saturday
1    2011  January       1  Saturday
2    2011  January       2  Saturday
3    2011  January       3  Saturday
4    2011  January       4  Saturday
```

5 Summarize Data, part 2

5.1 Descriptive Statistics

[ ]:
```
# shape
print('The dataset has %d rows and %d columns. \n' % (train.shape[0], train.
 ↪shape[1]))
```

```
The dataset has 10886 rows and 12 columns.
```

[ ]:
```
train.head()
```

[ ]:
```
   season holiday workingday weather  temp  humidity  windspeed  count  \
0  Spring      No         No   Clear  9.84        81        0.0     16
1  Spring      No         No   Clear  9.02        80        0.0     40
2  Spring      No         No   Clear  9.02        80        0.0     32
3  Spring      No         No   Clear  9.84        75        0.0     13
4  Spring      No         No   Clear  9.84        75        0.0      1

  dt_year dt_month dt_hour    dt_day
0    2011  January       0  Saturday
1    2011  January       1  Saturday
2    2011  January       2  Saturday
3    2011  January       3  Saturday
4    2011  January       4  Saturday
```

[ ]:
```
# info
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   season      10886 non-null  object
 1   holiday     10886 non-null  object
```

```
 2   workingday  10886 non-null  object
 3   weather     10886 non-null  object
 4   temp        10886 non-null  float64
 5   humidity    10886 non-null  int64
 6   windspeed   10886 non-null  float64
 7   count       10886 non-null  int64
 8   dt_year     10886 non-null  category
 9   dt_month    10886 non-null  category
 10  dt_hour     10886 non-null  category
 11  dt_day      10886 non-null  category
dtypes: category(4), float64(2), int64(2), object(4)
memory usage: 724.6+ KB
```

5.2 Numeric Features

[ ]:
```
# numeric features
numeric_features = list(train.select_dtypes(include=['int64', 'float']).columns)
train[numeric_features].describe()
```

[ ]:

	temp	humidity	windspeed	count
count	10886.00000	10886.000000	10886.000000	10886.000000
mean	20.23086	61.886460	12.799395	191.574132
std	7.79159	19.245033	8.164537	181.144454
min	0.82000	0.000000	0.000000	1.000000
25%	13.94000	47.000000	7.001500	42.000000
50%	20.50000	62.000000	12.998000	145.000000
75%	26.24000	77.000000	16.997900	284.000000
max	41.00000	100.000000	56.996900	977.000000

5.2.1 D'Agostino's K2 test

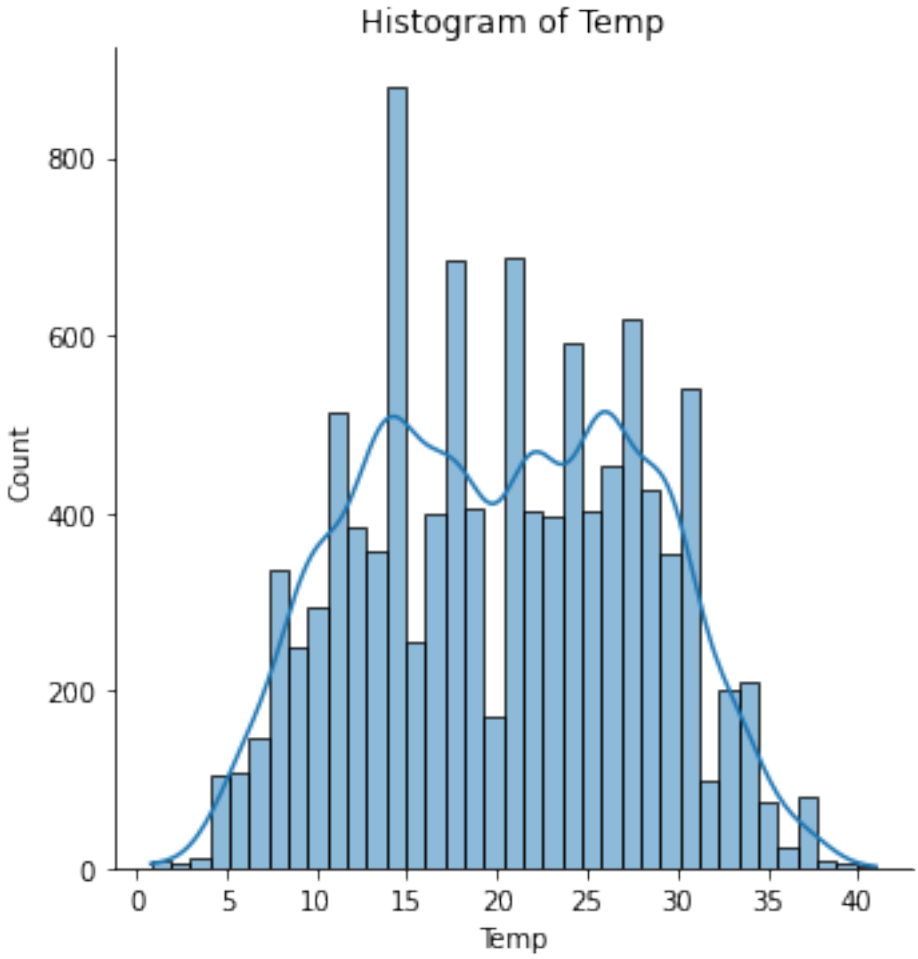
In statistics, D'Agostino's K2 test, named for Ralph D'Agostino, is a goodness-of-fit measure of departure from normality, that is the test aims to establish whether or not the given sample comes from a normally distributed population. The test is based on transformations of the sample kurtosis and skewness, and has power only against the alternatives that the distribution is skewed and/or kurtic.

https://en.wikipedia.org/wiki/D%27Agostino%27s_K-squared_test

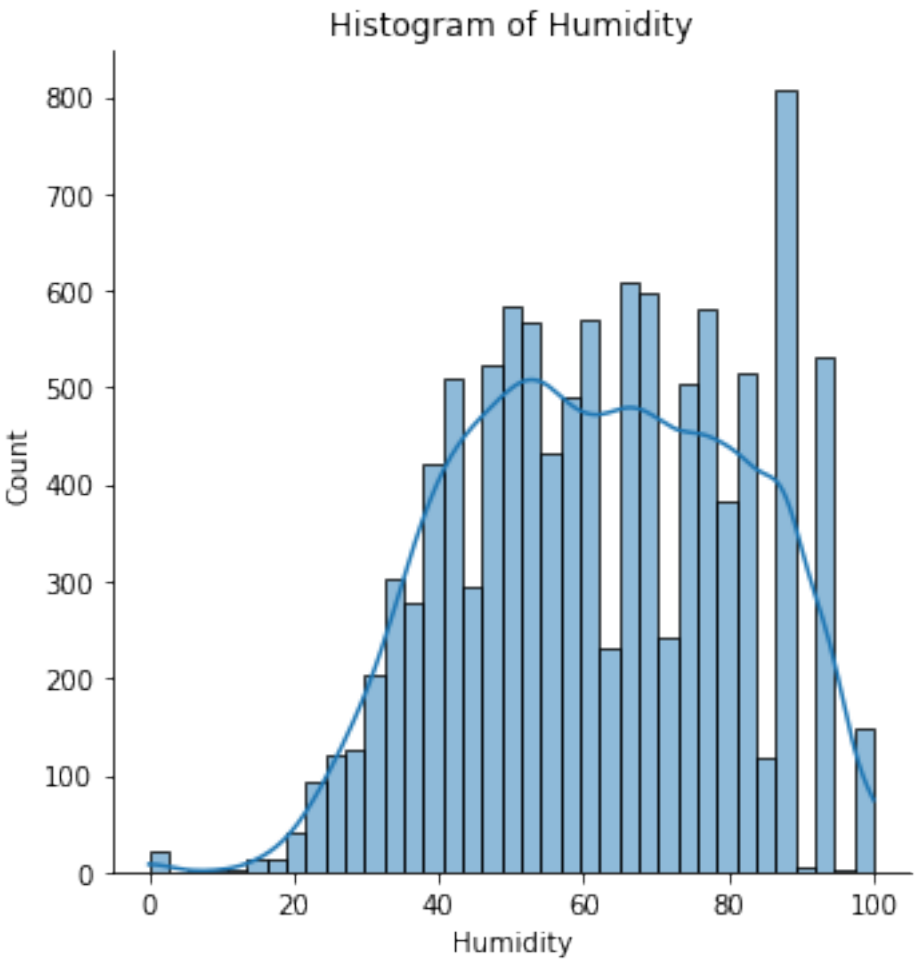
[ ]:
```
# import libaries
from scipy.stats import normaltest
import matplotlib.pyplot as plt
import seaborn as sns

# test for normality and graphs
for feature in numeric_features:
    sns.displot(train[feature], palette='colorblind', kde=True)
    title = 'Histogram of ' + feature.capitalize()
```

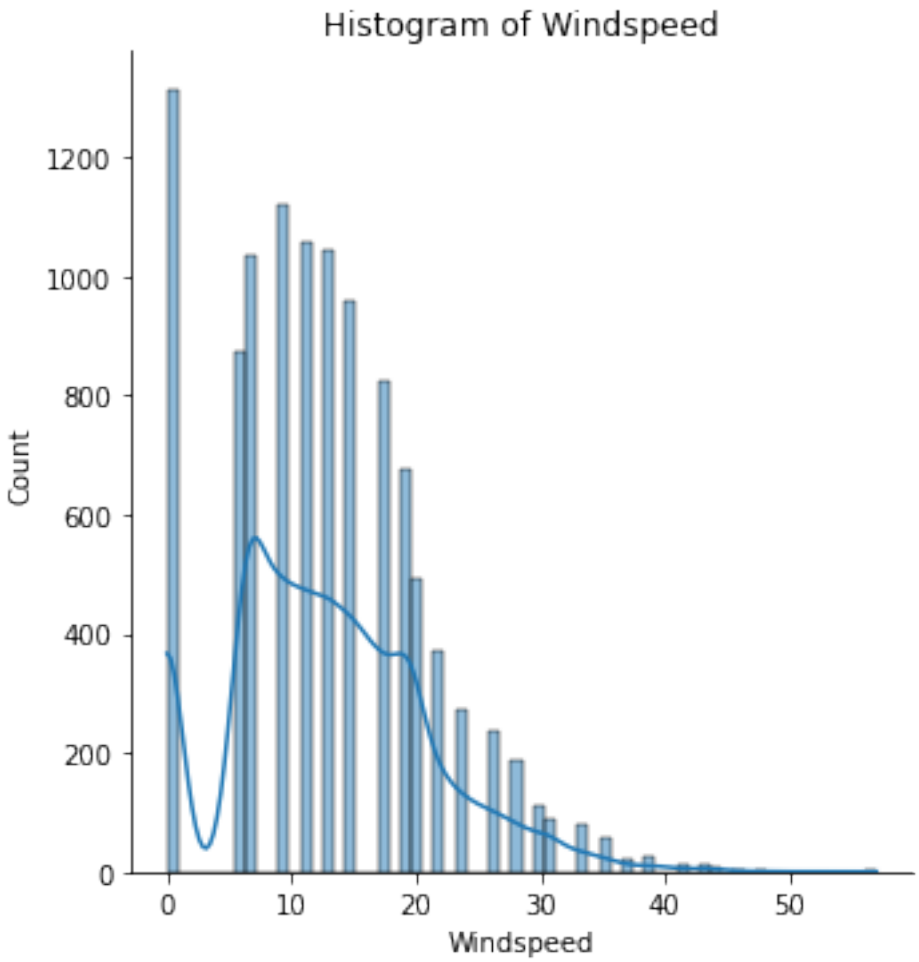
```
    plt.title(title)
    plt.xlabel(feature.capitalize())
    plt.show()
    if normaltest(train[feature])[1] >= 0.05:
        print('\'{}\' is normally distributed with a p-value of {:.4f}.'.
↪format(feature, normaltest(train[feature])[1]))
    else:
        print('\'{}\' is not normally distributed with a p-value of {:.4f}.'.
↪format(feature, normaltest(train[feature])[1]))
```



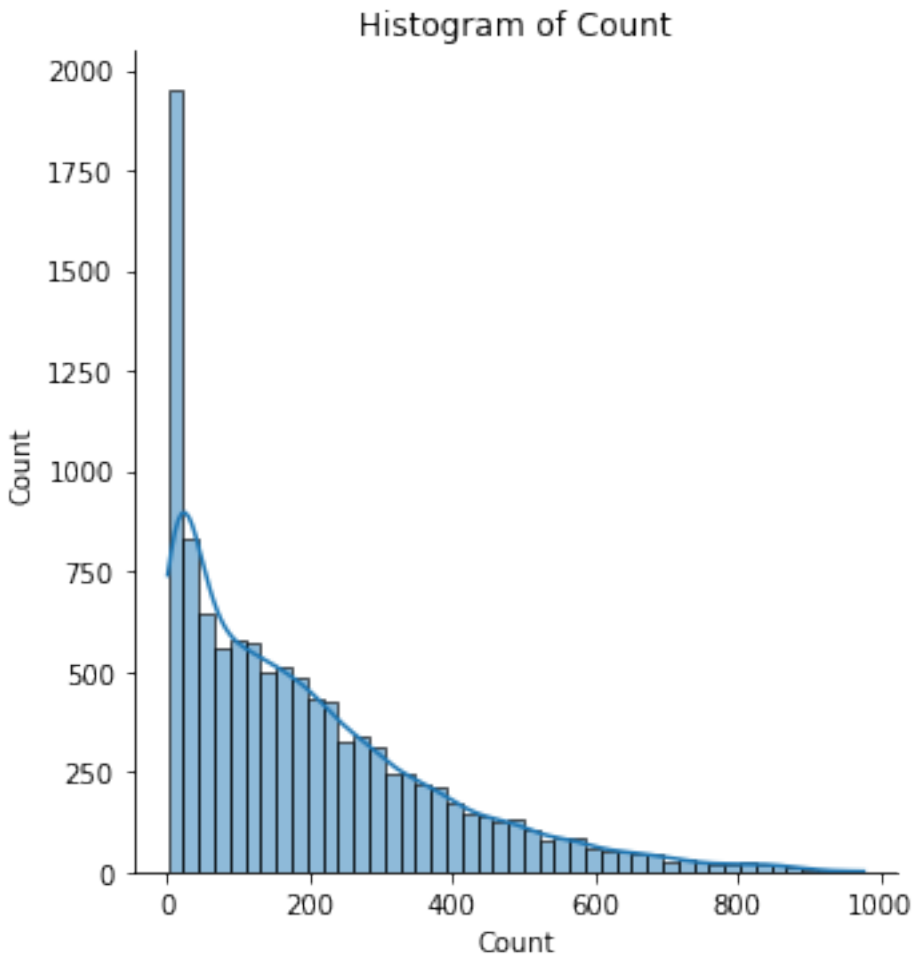

```
'temp' is not normally distributed with a p-value of 0.0000.
```




```
'humidity' is not normally distributed with a p-value of 0.0000.
```



'windspeed' is not normally distributed with a p-value of 0.0000.

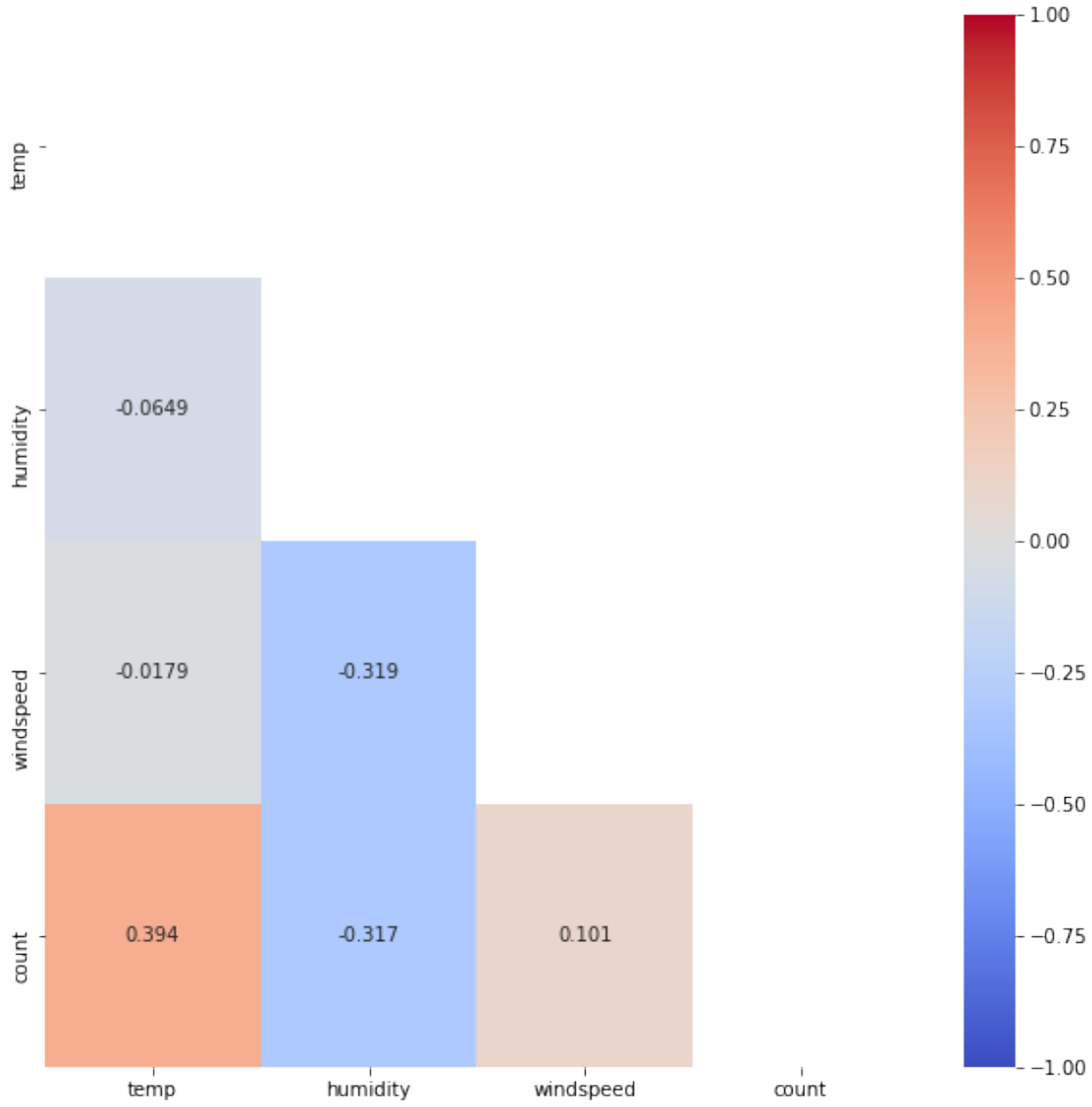



```
'count' is not normally distributed with a p-value of 0.0000.
```

5.2.2 Correlation

Windspeed has a very low correlation. Can consider dropping.

[ ]:
```
# correlation heatmap
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
matrix = np.triu(train.corr(method='pearson'))
plt.figure(figsize=(10,10))
sns.heatmap(train.corr(method='pearson'),annot=True, vmin=-1, vmax=1, center=␣
↪0, mask=matrix, cmap='coolwarm', fmt='.3g')
plt.show()
# correlation
train.corr()
```



[ ]:

	temp	humidity	windspeed	count
temp	1.000000	-0.064949	-0.017852	0.394454
humidity	-0.064949	1.000000	-0.318607	-0.317371
windspeed	-0.017852	-0.318607	1.000000	0.101369
count	0.394454	-0.317371	0.101369	1.000000

5.3 Categorical Features

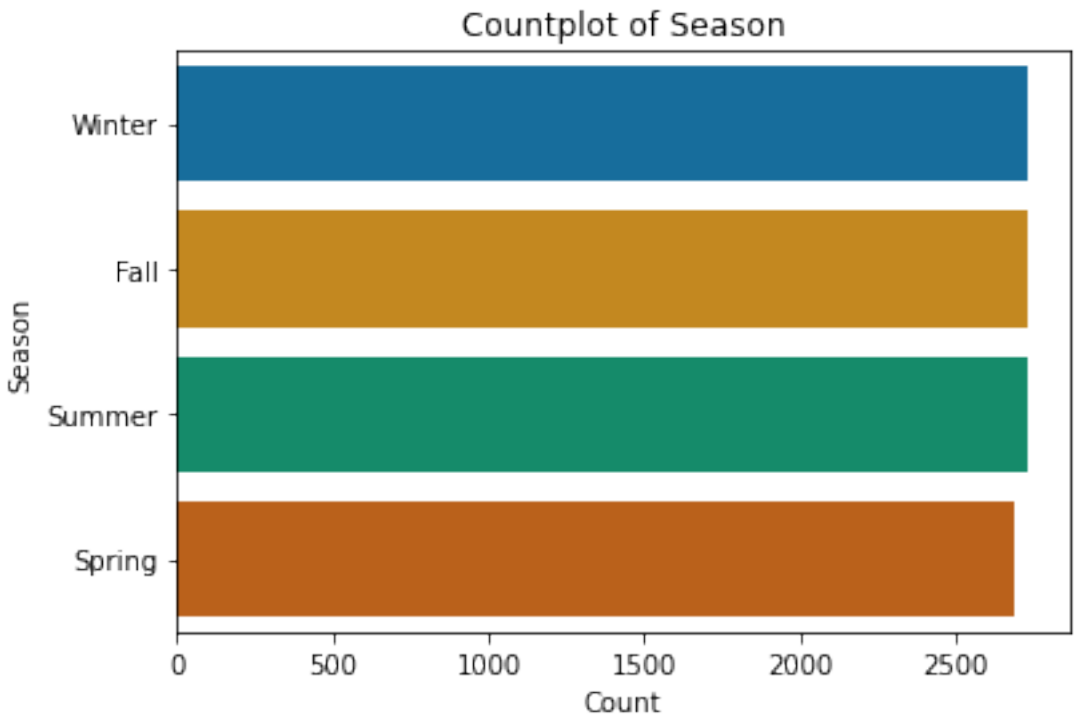
[ ]:
```
categorical_features = list(train.select_dtypes(include=['object', 'category']).
↪columns)
train[categorical_features].describe()
```

[ ]:

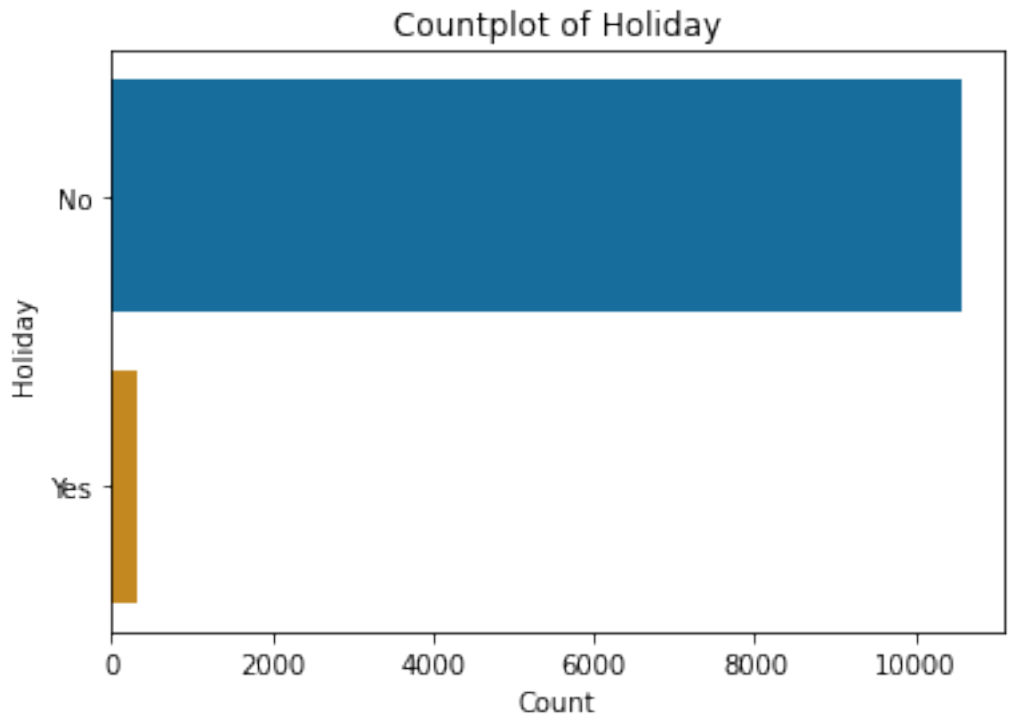
	season	holiday	workingday	weather	dt_year	dt_month	dt_hour	dt_day
count	10886	10886	10886	10886	10886	10886	10886	10886
unique	4	2	2	4	2	12	24	7
top	Winter	No	Yes	Clear	2012	August	12	Saturday
freq	2734	10575	7412	7192	5464	912	456	1584

5.3.1 Count Plots

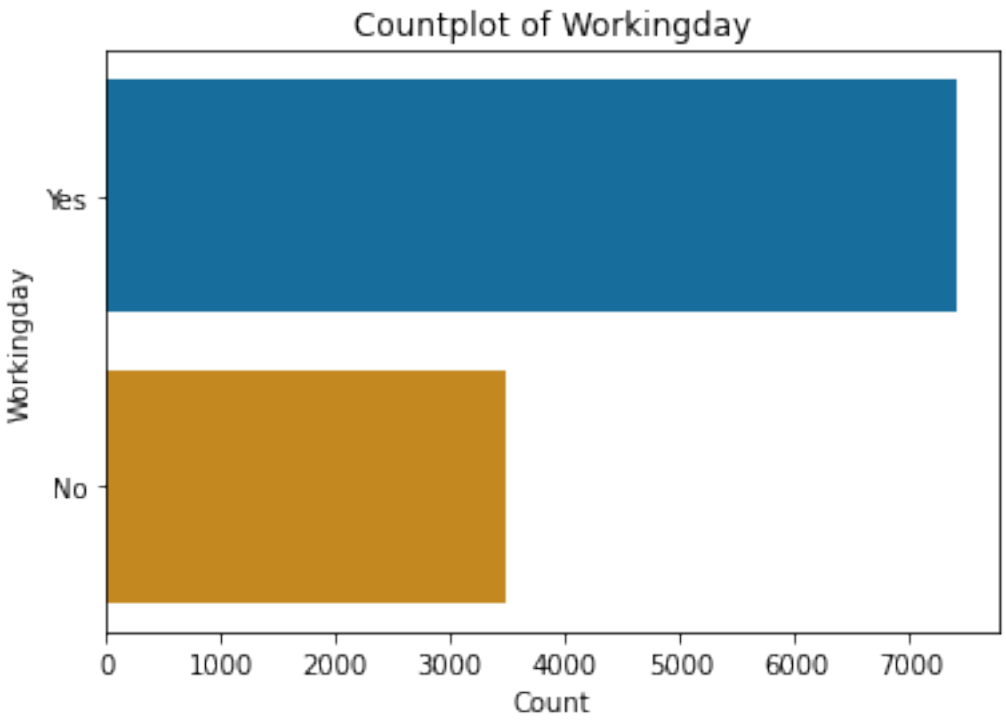
[ ]:
```
# import libraries
import seaborn as sns
import matplotlib.pyplot as plt
# count plots
for feature in categorical_features:
    if train[feature].nunique() < 15:
        sns.countplot(y=feature, data=train, palette='colorblind', order =␣
 ↪train[feature].value_counts().index)
        plt.title('Countplot of {}'.format(feature.capitalize()))
        plt.xlabel('Count')
        plt.ylabel(feature.capitalize())
        plt.show()
        print(pd.DataFrame(train[feature].value_counts()))
    else:
        sns.countplot(y=feature, data=train, palette='colorblind', order =␣
 ↪train[feature].value_counts().iloc[:10].index)
        plt.title('Countplot of Top 10 {}'.format(feature.capitalize()))
        plt.xlabel('Count')
        plt.ylabel(feature.capitalize())
        plt.show()
        print(pd.DataFrame(train[feature].value_counts().iloc[:10]))
```



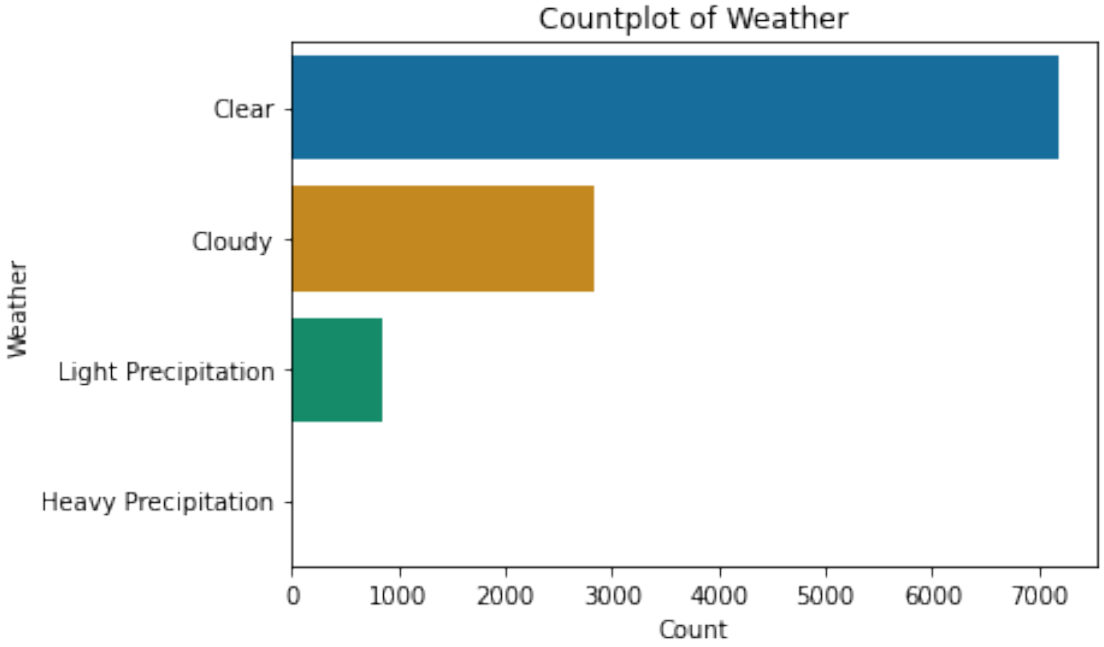
	season
Winter	2734
Fall	2733
Summer	2733
Spring	2686



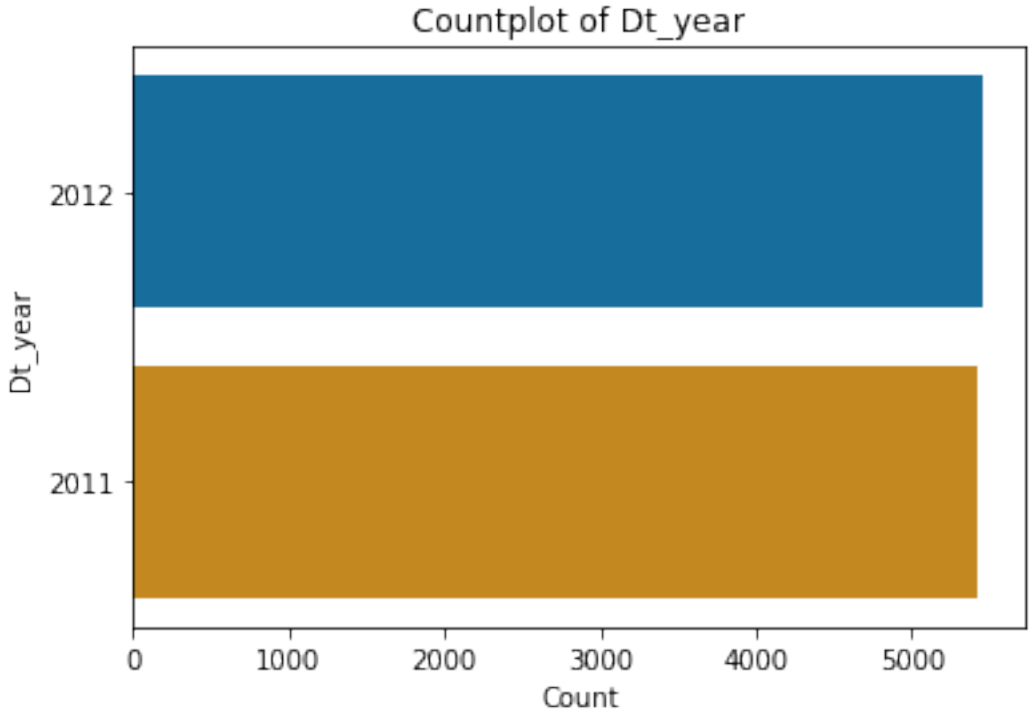
	holiday
No	10575
Yes	311



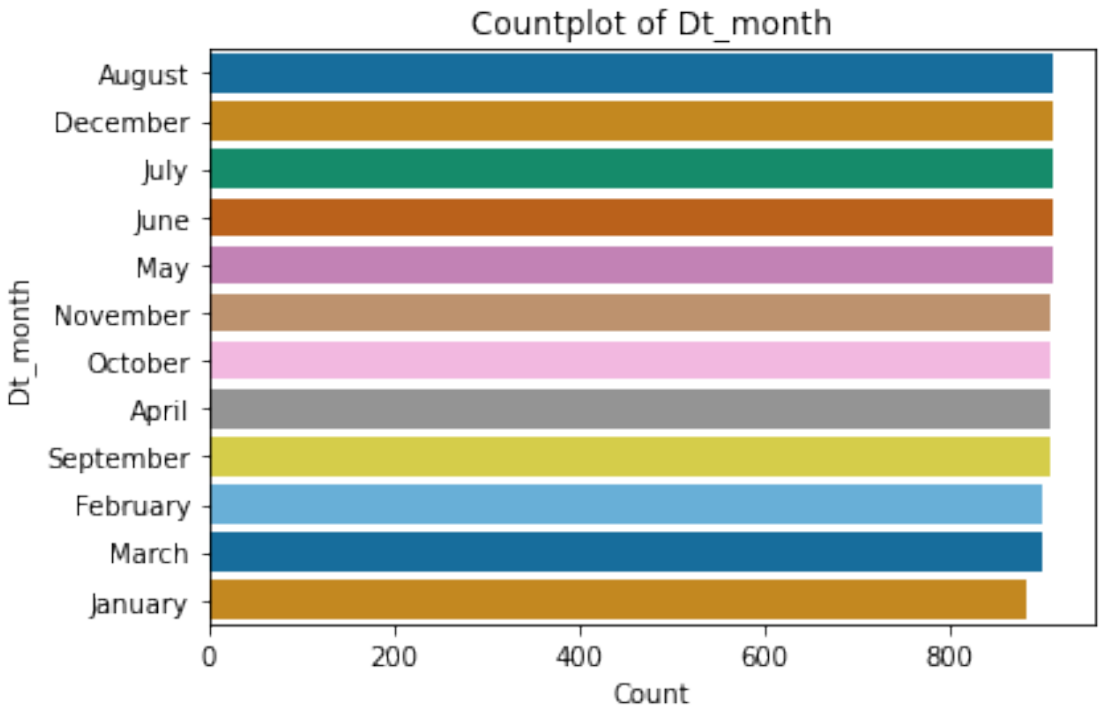
	workingday
Yes	7412
No	3474



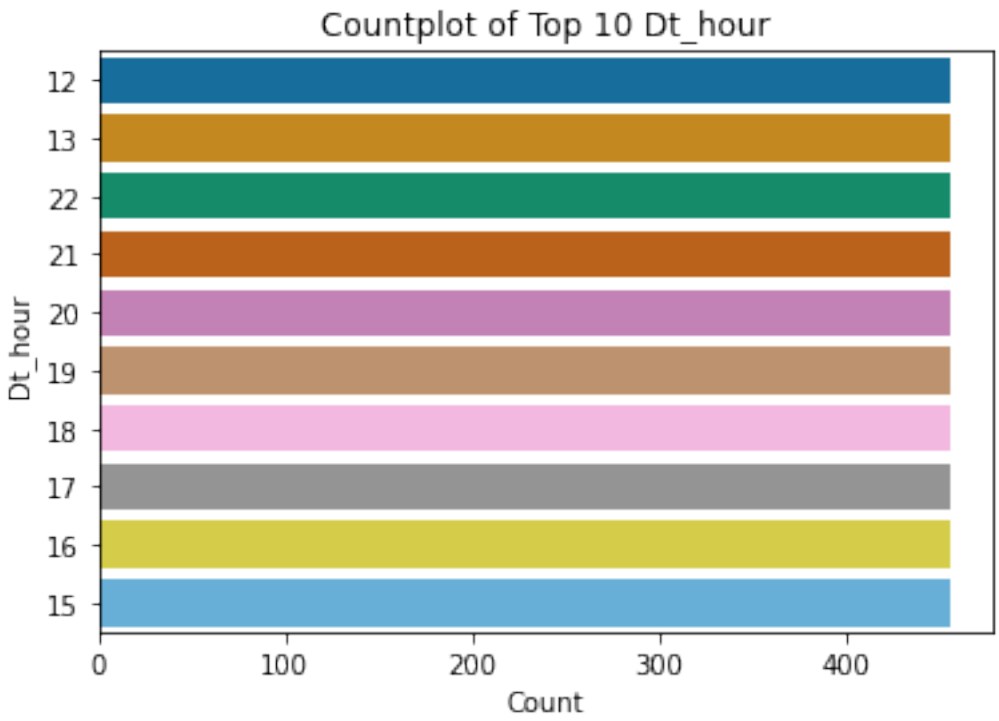
	weather
Clear	7192
Cloudy	2834
Light Precipitation	859
Heavy Precipitation	1



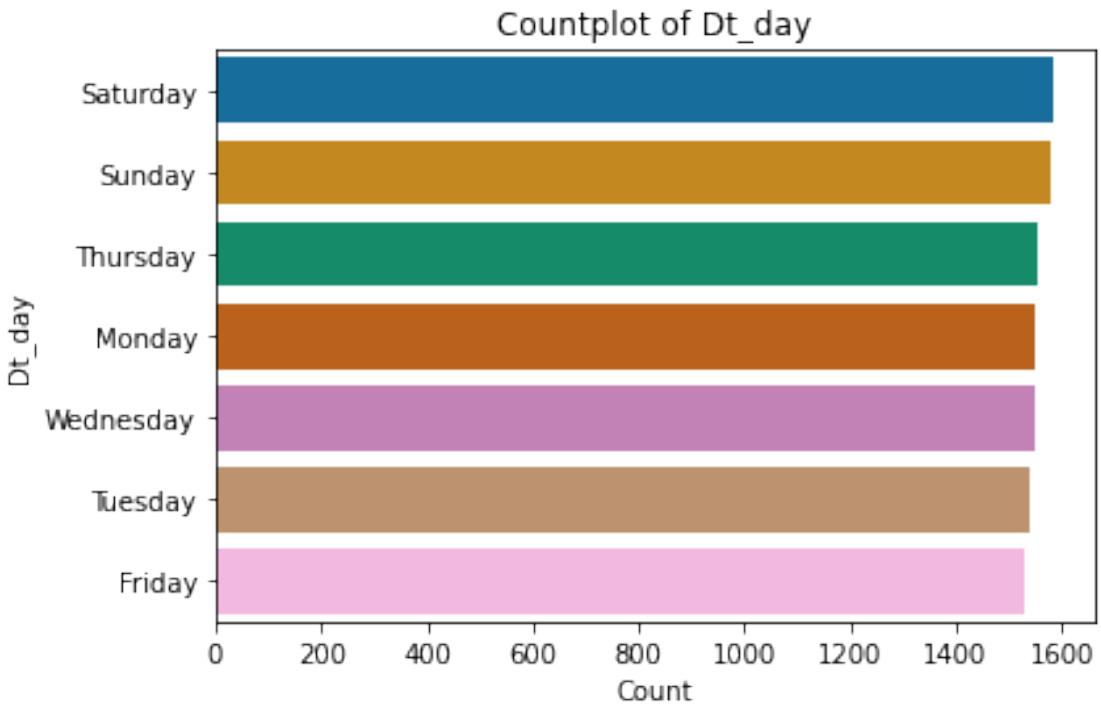
	dt_year
2012	5464
2011	5422



	dt_month
August	912
December	912
July	912
June	912
May	912
November	911
October	911
April	909
September	909
February	901
March	901
January	884



	dt_hour
12	456
13	456
22	456
21	456
20	456
19	456
18	456
17	456
16	456
15	456



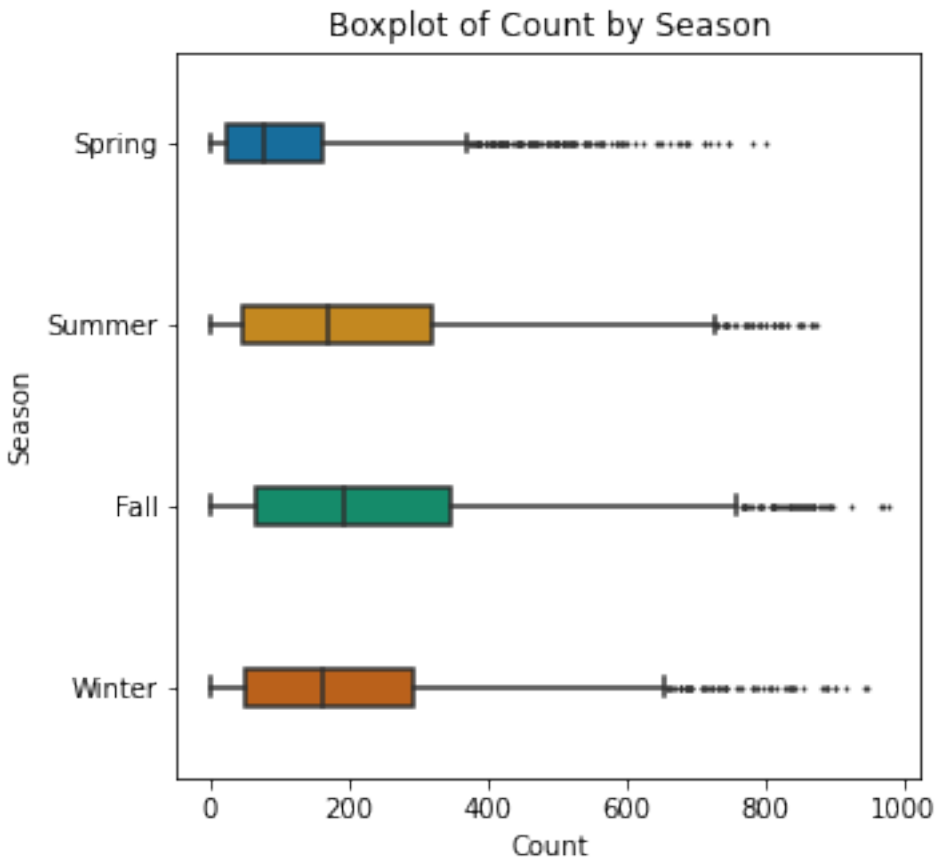
	dt_day
Saturday	1584
Sunday	1579
Thursday	1553
Monday	1551
Wednesday	1551
Tuesday	1539
Friday	1529

5.3.2 ANOVA

[ ]:
```
# import libraries
import statsmodels.api as sm
from statsmodels.formula.api import ols
# list of independent features
independent_features = []
# Function for Anova
def aov(cat):
    # features
    num = 'count'
    cat = cat
    # create formula
    formula = num + ' ~ ' + cat
    # create ols
```

```
    mod = ols(formula, data=train).fit()
    # aov table
    aov_table = sm.stats.anova_lm(mod, typ=2)
    # plot boxplot
    if train[cat].nunique() > 15:
        plt.figure(figsize=(7,10))
        sns.boxplot(y=cat, x=num, palette='colorblind', data=train, width=0.2,␣
↪fliersize=1)
        plt.title('Boxplot of {} by {}'.format(num.capitalize(), cat.
↪capitalize()))
        plt.xlabel(num.capitalize())
        plt.ylabel(cat.capitalize())
        plt.show()
    else:
        plt.figure(figsize=(5,5))
        sns.boxplot(y=cat, x=num, palette='colorblind', data=train, width=0.2,␣
↪fliersize=1)
        plt.title('Boxplot of {} by {}'.format(num.capitalize(), cat.
↪capitalize()))
        plt.xlabel(num.capitalize())
        plt.ylabel(cat.capitalize())
        plt.show()
    # display mean of group
    display('Means by Group:')
    display(train.groupby(cat, as_index=False)[num].mean())
    # display aov results
    display('AOV Results:')
    display(aov_table)
    # display pairwise comparison
    display('Pairwise Comparisons:')
    pair_t = mod.t_test_pairwise(cat)
    display(pair_t.result_frame)
    # get p-value of F and compare
    PR_F = sm.stats.anova_lm(mod, typ=2)['PR(>F)'][0]
    if PR_F < 0.10:
        print('The features of \'{}\' and \'{}\' are {} with a p-value of {:.
↪4f}.'.format(num, cat, 'dependent', PR_F))
    else:
        print('The features of \'{}\' and \'{}\' are {} with a p-value of {:.
↪4f}.'.format(num, cat, 'independent', PR_F))
        independent_features.append(cat)
```

[ ]:
```
for feature in categorical_features:
    aov(feature)
```



'Means by Group:'

	season	count
0	Fall	234.417124
1	Spring	116.343261
2	Summer	215.251372
3	Winter	198.988296

'AOV Results:'

	sum_sq	df	F	PR(>F)
season	2.190083e+07	3.0	236.946711	6.164843e-149
Residual	3.352721e+08	10882.0	NaN	NaN

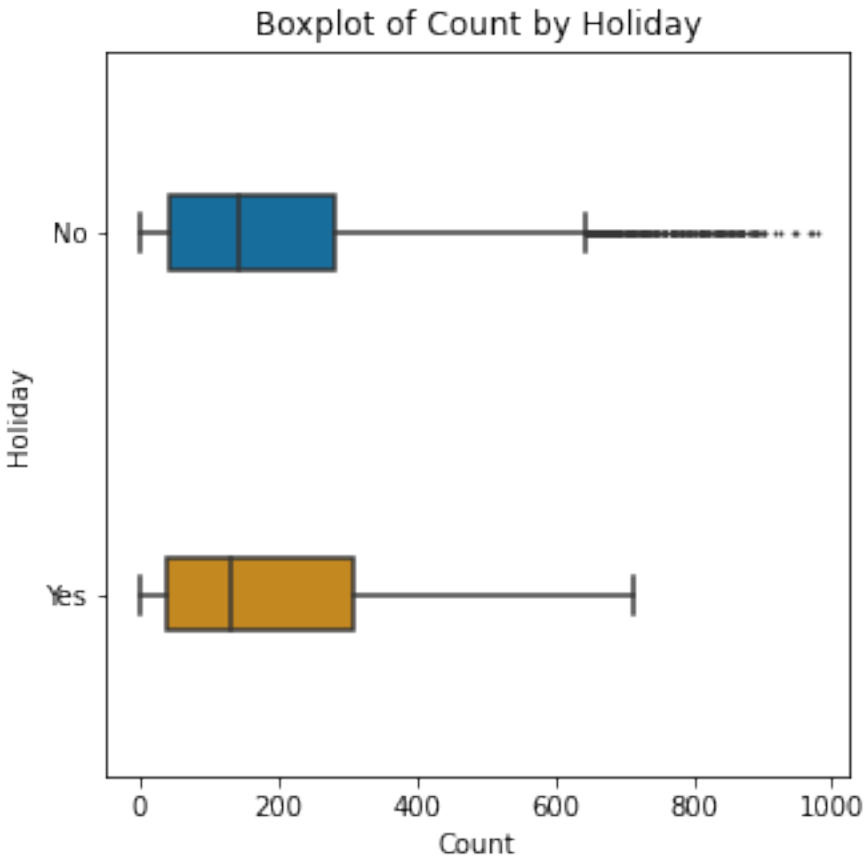
'Pairwise Comparisons:'

	coef	std err	t	P>|t|	Conf. Int. Low	\
Spring-Fall	-118.073863	4.769041	-24.758406	1.063174e-131	-127.422052	
Summer-Fall	-19.165752	4.748315	-4.036327	5.466633e-05	-28.473313	
Winter-Fall	-35.428829	4.747881	-7.462030	9.169927e-14	-44.735539	
Summer-Spring	98.908111	4.769041	20.739621	9.756471e-94	89.559922	
Winter-Spring	82.645034	4.768609	17.331057	2.127949e-66	73.297693	

```
Winter-Summer  -16.263077  4.747881  -3.425334   6.163118e-04      -25.569787
```

```
               Conf. Int. Upp.     pvalue-hs  reject-hs
Spring-Fall         -108.725674  0.000000e+00       True
Summer-Fall           -9.858190  1.093297e-04       True
Winter-Fall          -26.122118  2.751133e-13       True
Summer-Spring        108.256300  0.000000e+00       True
Winter-Spring         91.992376  0.000000e+00       True
Winter-Summer         -6.956366  6.163118e-04       True
```

```
The features of 'count' and 'season' are dependent with a p-value of 0.0000.
```




```
'Means by Group:'
```

```
  holiday       count
0      No  191.741655
1     Yes  185.877814
```

```
'AOV Results:'
```

```
                sum_sq       df          F    PR(>F)
holiday   1.038812e+04      1.0  0.316563  0.573692
```

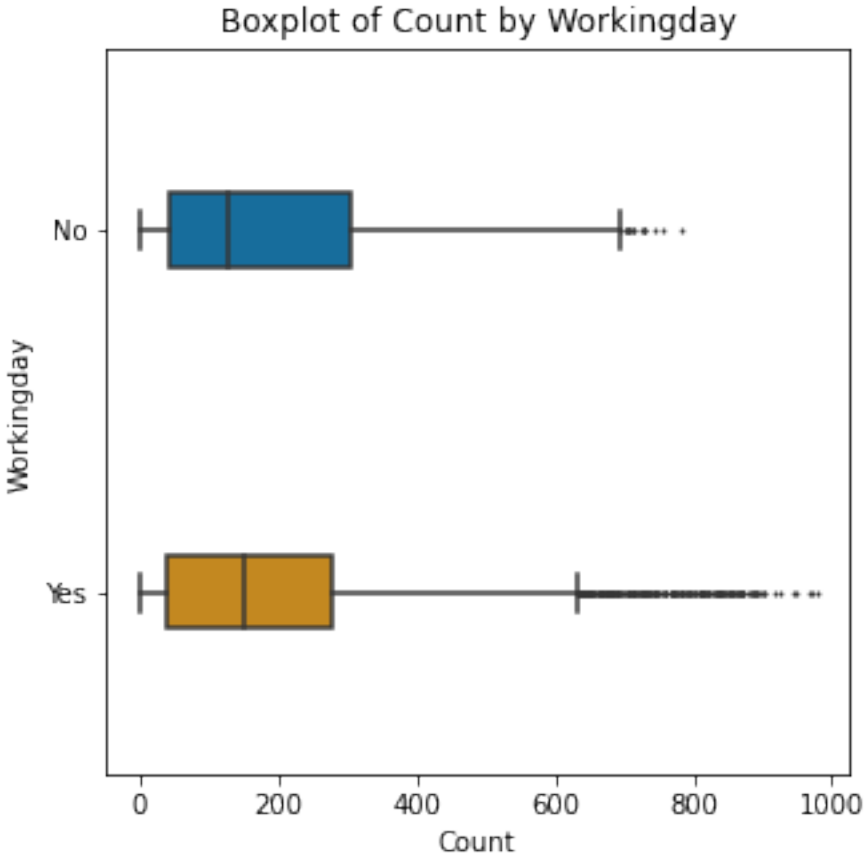
```
Residual  3.571625e+08  10884.0       NaN       NaN
```

'Pairwise Comparisons:'

```
            coef    std err         t     P>|t|  Conf. Int. Low  \
Yes-No -5.863841  10.422033 -0.562639  0.573692      -26.292923

        Conf. Int. Upp.  pvalue-hs  reject-hs
Yes-No         14.56524   0.573692      False
```

The features of 'count' and 'holiday' are independent with a p-value of 0.5737.



'Means by Group:'

```
  workingday       count
0         No  188.506621
1        Yes  193.011873
```

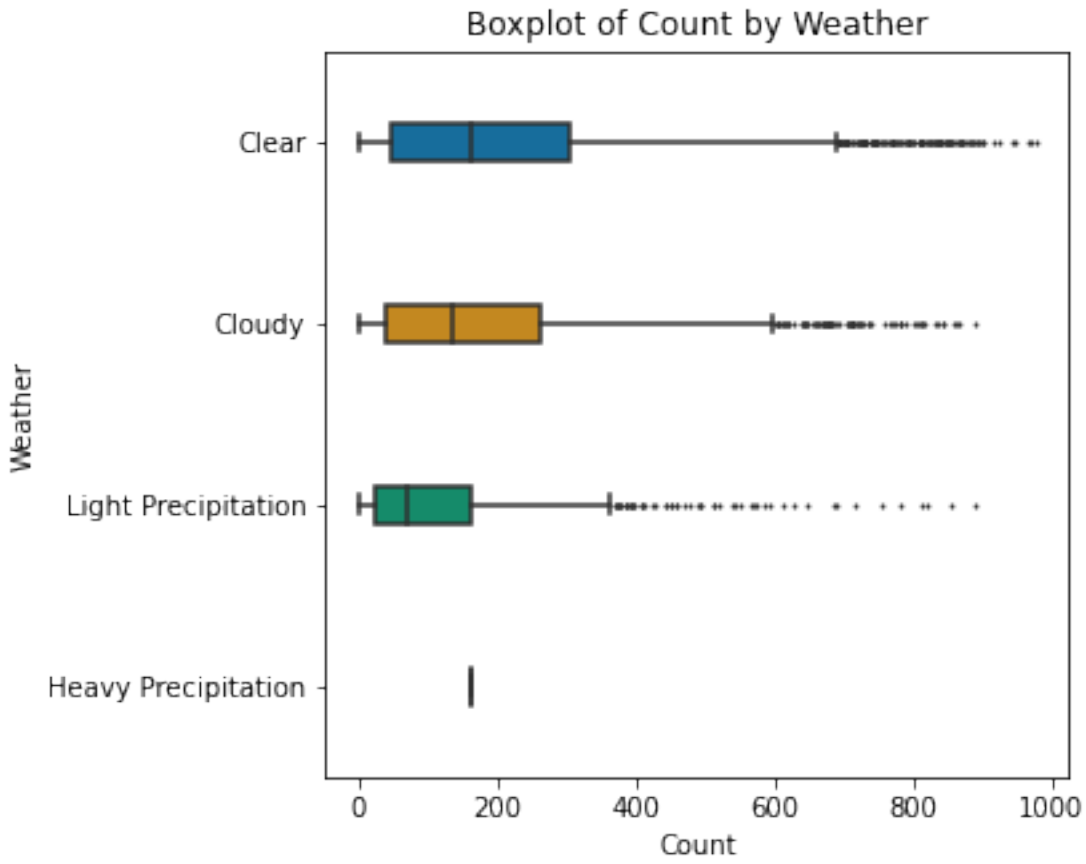
'AOV Results:'

```
                 sum_sq       df         F    PR(>F)
workingday  4.801037e+04      1.0  1.463199  0.226448
Residual    3.571249e+08  10884.0       NaN       NaN
```

'Pairwise Comparisons:'

	coef	std err	t	P>|t|	Conf. Int. Low	Conf. Int. Upp.	pvalue-hs	reject-hs
Yes-No	4.505252	3.724495	1.209628	0.226448	-2.795435	11.805939	0.226448	False

The features of 'count' and 'workingday' are independent with a p-value of 0.2264.



'Means by Group:'

	weather	count
0	Clear	205.236791
1	Cloudy	178.955540
2	Heavy Precipitation	164.000000
3	Light Precipitation	118.846333

'AOV Results:'

	sum_sq	df	F	PR(>F)
weather	6.338070e+06	3.0	65.530241	5.482069e-42

```
Residual  3.508348e+08  10882.0        NaN           NaN
```

'Pairwise Comparisons:'

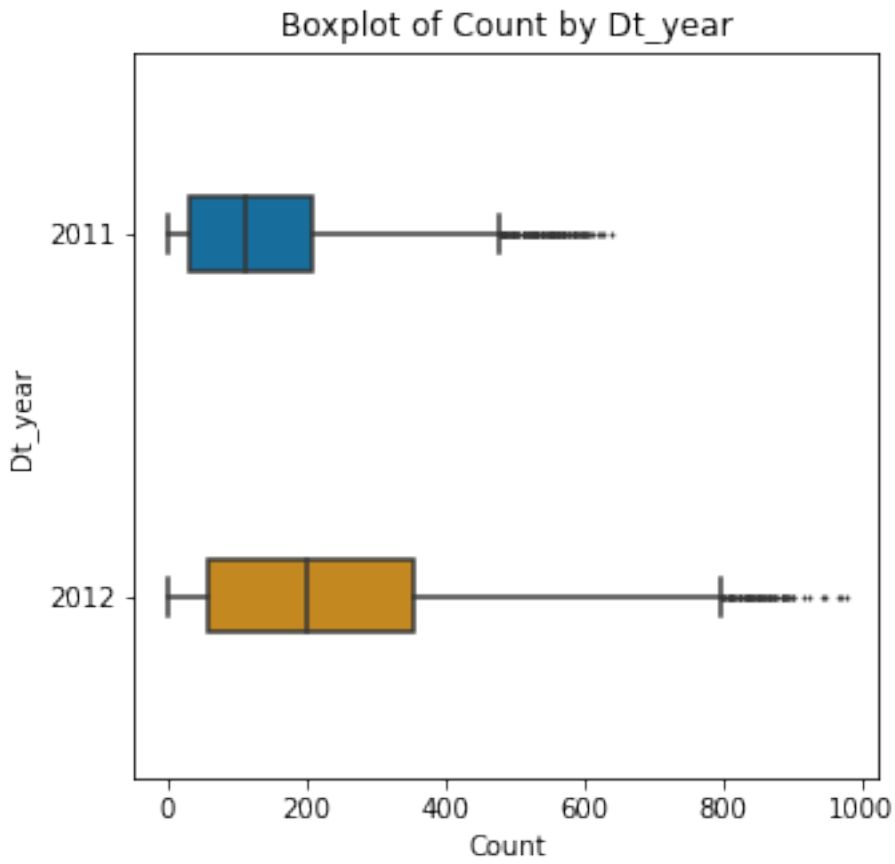
	coef	std err	t	\
Cloudy-Clear	-26.281251	3.982319	-6.599483	
Heavy Precipitation-Clear	-41.236791	179.567274	-0.229645	
Light Precipitation-Clear	-86.390458	6.481873	-13.328009	
Heavy Precipitation-Cloudy	-14.955540	179.586467	-0.083278	
Light Precipitation-Cloudy	-60.109207	6.993429	-8.595098	
Light Precipitation-Heavy Precipitation	-45.153667	179.659275	-0.251329	

	P>|t|	Conf. Int. Low	\
Cloudy-Clear	4.317735e-11	-34.087322	
Heavy Precipitation-Clear	8.183717e-01	-393.221331	
Light Precipitation-Clear	3.285377e-40	-99.096108	
Heavy Precipitation-Cloudy	9.336323e-01	-366.977702	
Light Precipitation-Cloudy	9.457082e-18	-73.817600	
Light Precipitation-Heavy Precipitation	8.015642e-01	-397.318545	

	Conf. Int. Upp.	pvalue-hs	\
Cloudy-Clear	-18.475180	1.727094e-10	
Heavy Precipitation-Clear	310.747749	9.921862e-01	
Light Precipitation-Clear	-73.684808	0.000000e+00	
Heavy Precipitation-Cloudy	337.066622	9.921862e-01	
Light Precipitation-Cloudy	-46.400814	0.000000e+00	
Light Precipitation-Heavy Precipitation	307.011211	9.921862e-01	

	reject-hs
Cloudy-Clear	True
Heavy Precipitation-Clear	False
Light Precipitation-Clear	True
Heavy Precipitation-Cloudy	False
Light Precipitation-Cloudy	True
Light Precipitation-Heavy Precipitation	False

The features of 'count' and 'weather' are dependent with a p-value of 0.0000.



'Means by Group:'

	dt_year	count
0	2011	144.223349
1	2012	238.560944

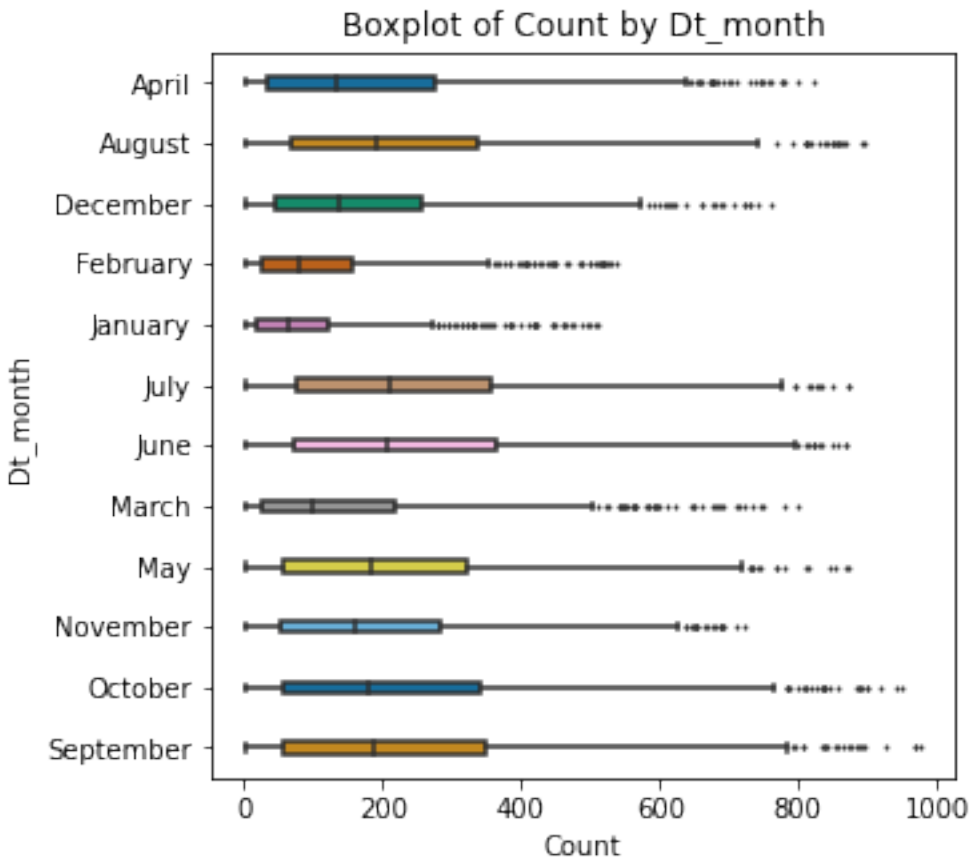
'AOV Results:'

	sum_sq	df	F	PR(>F)
dt_year	2.421985e+07	1.0	791.729805	3.242014e-168
Residual	3.329531e+08	10884.0	NaN	NaN

'Pairwise Comparisons:'

	coef	std err	t	P>|t|	Conf. Int. Low	Conf. Int. Upp.	pvalue-hs	reject-hs
2012-2011	94.337595	3.352712	28.137694	3.242014e-168	87.765669	100.909521	0.0	True

The features of 'count' and 'dt_year' are dependent with a p-value of 0.0000.



'Means by Group:'

	dt_month	count
0	April	184.160616
1	August	234.118421
2	December	175.614035
3	February	110.003330
4	January	90.366516
5	July	235.325658
6	June	242.031798
7	March	148.169811
8	May	219.459430
9	November	193.677278
10	October	227.699232
11	September	233.805281

'AOV Results:'

	sum_sq	df	F	PR(>F)
dt_month	2.627121e+07	11.0	78.483391	3.967012e-171
Residual	3.309017e+08	10874.0	NaN	NaN

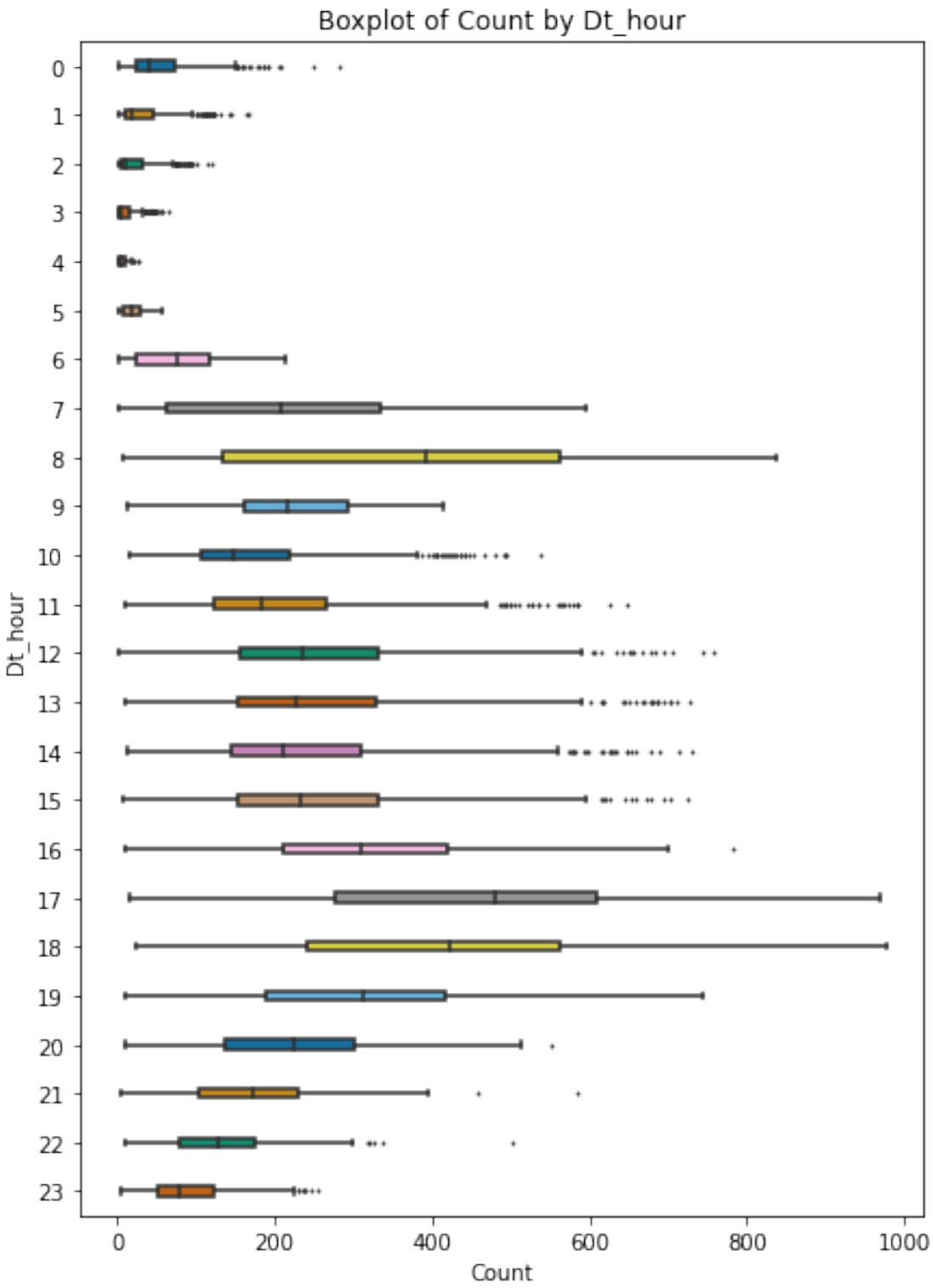
'Pairwise Comparisons:'

	coef	std err	t	P>|t|	\
August-April	49.957805	8.175804	6.110446	1.027556e-09	
December-April	-8.546581	8.175804	-1.045351	2.958842e-01	
February-April	-74.157286	8.200678	-9.042823	1.785758e-19	
January-April	-93.794100	8.240184	-11.382525	7.538009e-30	
July-April	51.165042	8.175804	6.258106	4.043530e-10	
…	…	…	…	…	
October-May	8.239802	8.171308	1.008382	3.132934e-01	
September-May	14.345851	8.175804	1.754672	7.934365e-02	
October-November	34.021954	8.173549	4.162446	3.172824e-05	
September-November	40.128003	8.178043	4.906798	9.391786e-07	
September-October	6.106049	8.178043	0.746639	4.552974e-01	

	Conf. Int. Low	Conf. Int. Upp.	pvalue-hs	reject-hs
August-April	33.931741	65.983869	3.493692e-08	True
December-April	-24.572645	7.479483	9.789096e-01	False
February-April	-90.232110	-58.082463	0.000000e+00	True
January-April	-109.946362	-77.641839	0.000000e+00	True
July-April	35.138977	67.191106	1.415235e-08	True
…	…	…	…	…
October-May	-7.777450	24.257053	9.789096e-01	False
September-May	-1.680214	30.371915	6.856845e-01	False
October-November	18.000310	50.043598	6.977888e-04	True
September-November	24.097548	56.158457	2.441836e-05	True
September-October	-9.924406	22.136503	9.789096e-01	False

[66 rows x 8 columns]

The features of 'count' and 'dt_month' are dependent with a p-value of 0.0000.



'Means by Group:'

```
   dt_hour       count
0        0   55.138462
1        1   33.859031
2        2   22.899554
3        3   11.757506
4        4    6.407240
5        5   19.767699
6        6   76.259341
7        7  213.116484
8        8  362.769231
9        9  221.780220
10      10  175.092308
11      11  210.674725
12      12  256.508772
13      13  257.787281
14      14  243.442982
15      15  254.298246
16      16  316.372807
17      17  468.765351
18      18  430.859649
19      19  315.278509
20      20  228.517544
21      21  173.370614
22      22  133.576754
23      23   89.508772
```

'AOV Results:'

```
                sum_sq       df           F  PR(>F)
dt_hour   1.845339e+08     23.0  504.799696     0.0
Residual  1.726390e+08  10862.0         NaN     NaN
```

'Pairwise Comparisons:'

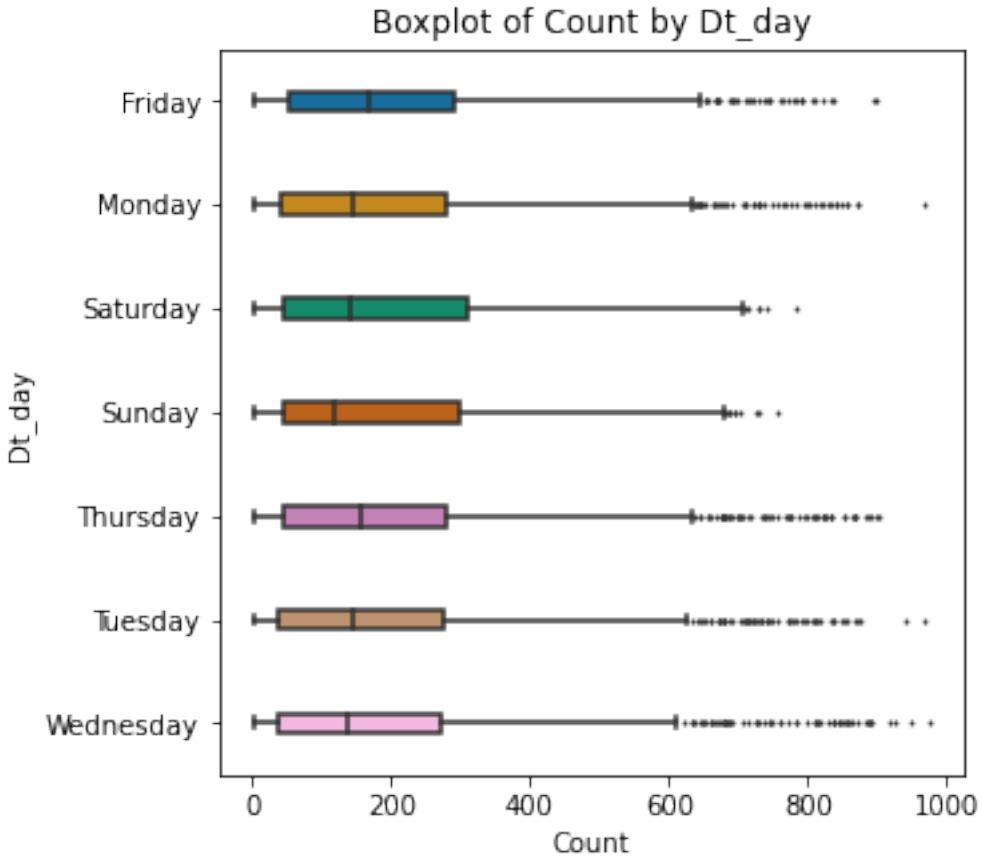
```
             coef   std err          t         P>|t|  Conf. Int. Low  \
1-0    -21.279431  8.363016  -2.544469  1.095814e-02      -37.672467
2-0    -32.238908  8.391001  -3.842081  1.226897e-04      -48.686800
3-0    -43.380956  8.463918  -5.125399  3.020133e-07      -59.971778
4-0    -48.731222  8.419649  -5.787797  7.328880e-09      -65.235270
5-0    -35.370762  8.372272  -4.224751  2.411578e-05      -51.781943
…            …        …         …           …                …
22-20  -94.940789  8.349244 -11.371183  8.575141e-30     -111.306831
23-20 -139.008772  8.349244 -16.649264  1.765690e-61     -155.374814
22-21  -39.793860  8.349244  -4.766163  1.902030e-06      -56.159902
23-21  -83.861842  8.349244 -10.044243  1.234910e-23     -100.227884
23-22  -44.067982  8.349244  -5.278080  1.330609e-07      -60.434024

       Conf. Int. Upp.     pvalue-hs  reject-hs
1-0          -4.886394  2.491010e-01      False
2-0         -15.791016  4.163016e-03       True
```

```
3-0          -26.790133  1.510055e-05       True
4-0          -32.227174  4.397327e-07       True
5-0          -18.959582  9.400848e-04       True
…                  …           …          …
22-20        -78.574747  0.000000e+00       True
23-20       -122.642730  0.000000e+00       True
22-21        -23.427818  8.748964e-05       True
23-21        -67.495800  0.000000e+00       True
23-22        -27.701940  7.052203e-06       True

[276 rows x 8 columns]
```

The features of 'count' and 'dt_hour' are dependent with a p-value of 0.0000.



'Means by Group:'

```
      dt_day       count
0     Friday  197.844343
1     Monday  190.390716
2   Saturday  196.665404
3     Sunday  180.839772
```

```
4   Thursday  197.296201
5    Tuesday  189.723847
6  Wednesday  188.411348
```

```
'AOV Results:'
```

```
               sum_sq       df         F    PR(>F)
dt_day    3.569194e+05      6.0  1.813692  0.092198
Residual  3.568160e+08  10879.0       NaN       NaN
```

```
'Pairwise Comparisons:'
```

```
                        coef   std err         t     P>|t|  Conf. Int. Low  \
Monday-Friday       -7.453627  6.526696 -1.142021  0.253470      -20.247140
Saturday-Friday     -1.178939  6.492858 -0.181575  0.855920      -13.906122
Sunday-Friday      -17.004571  6.497905 -2.616931  0.008885      -29.741648
Thursday-Friday     -0.548142  6.524609 -0.084011  0.933049      -13.337564
Tuesday-Friday      -8.120496  6.539316 -1.241796  0.214339      -20.938745
Wednesday-Friday    -9.432995  6.526696 -1.445294  0.148404      -22.226508
Saturday-Monday      6.274688  6.469384  0.969905  0.332115       -6.406483
Sunday-Monday       -9.550944  6.474450 -1.475175  0.140195      -22.242044
Thursday-Monday      6.905485  6.501250  1.062178  0.288178       -5.838149
Tuesday-Monday      -0.666869  6.516009 -0.102343  0.918486      -13.439434
Wednesday-Monday    -1.979368  6.503345 -0.304362  0.760858      -14.727108
Sunday-Saturday    -15.825632  6.440337 -2.457268  0.014015      -28.449866
Thursday-Saturday    0.630797  6.467279  0.097537  0.922302      -12.046248
Tuesday-Saturday    -6.941557  6.482115 -1.070878  0.284248      -19.647684
Wednesday-Saturday  -8.254057  6.469384 -1.275864  0.202031      -20.935228
Thursday-Sunday     16.456429  6.472346  2.542575  0.011018        3.769452
Tuesday-Sunday       8.884075  6.487171  1.369484  0.170876       -3.831962
Wednesday-Sunday     7.571576  6.474450  1.169455  0.242246       -5.119525
Tuesday-Thursday    -7.572354  6.513919 -1.162488  0.245063      -20.340822
Wednesday-Thursday  -8.884853  6.501250 -1.366638  0.171767      -21.628488
Wednesday-Tuesday   -1.312499  6.516009 -0.201427  0.840369      -14.085064

                    Conf. Int. Upp.  pvalue-hs  reject-hs
Monday-Friday              5.339885   0.964162      False
Saturday-Friday           11.548245   0.999896      False
Sunday-Friday             -4.267494   0.170898      False
Thursday-Friday           12.241280   0.999896      False
Tuesday-Friday             4.697753   0.957557      False
Wednesday-Friday           3.360517   0.934842      False
Saturday-Monday           18.955860   0.964162      False
Sunday-Monday              3.140157   0.934052      False
Thursday-Monday           19.649120   0.964162      False
Tuesday-Monday            12.105696   0.999896      False
Wednesday-Monday          10.768371   0.999813      False
Sunday-Saturday           -3.201398   0.235226      False
Thursday-Saturday         13.307842   0.999896      False
Tuesday-Saturday           5.764569   0.964162      False
```

```
Wednesday-Saturday          4.427115   0.957557      False
Thursday-Sunday            29.143406   0.198744      False
Tuesday-Sunday             21.600111   0.950122      False
Wednesday-Sunday           20.262676   0.964162      False
Tuesday-Thursday            5.196114   0.964162      False
Wednesday-Thursday          3.858781   0.950122      False
Wednesday-Tuesday          11.460066   0.999896      False
```

The features of 'count' and 'dt_day' are dependent with a p-value of 0.0922.

[ ]:
```
# features with little/no predictive power
print('The following features have little/no predictive power: {}.'.format(', '.
 ↪join(independent_features)))
train = train.drop(['windspeed', 'dt_month'], axis='columns')
test = test.drop(['windspeed', 'dt_month'], axis='columns')
train.head()
```

The following features have little/no predictive power: holiday, workingday.

[ ]:

	season	holiday	workingday	weather	temp	humidity	count	dt_year	dt_hour	\
0	Spring	No	No	Clear	9.84	81	16	2011	0	
1	Spring	No	No	Clear	9.02	80	40	2011	1	
2	Spring	No	No	Clear	9.02	80	32	2011	2	
3	Spring	No	No	Clear	9.84	75	13	2011	3	
4	Spring	No	No	Clear	9.84	75	1	2011	4	

	dt_day
0	Saturday
1	Saturday
2	Saturday
3	Saturday
4	Saturday

6 Model Setup

[ ]:
```
# Split into X, y
X = train.drop(['count'], axis='columns')
y = train['count']

# Train & Validation Sets
from sklearn.model_selection import train_test_split
X_train, X_validate, y_train, y_validate = train_test_split(X, y, test_size=0.
 ↪30)

# Separate Columns by type
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object', 'category']).columns
```

[ ]:
```
# import libraries
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, PowerTransformer,␣
 ↪OneHotEncoder, RobustScaler
from sklearn.impute import SimpleImputer

# Categorical Feature Pipeline (OneHot)
categorical_pipeline_steps = []
categorical_pipeline_steps.append(('onehot',␣
 ↪OneHotEncoder(handle_unknown='ignore')))
categorical_pipeline = Pipeline(steps=categorical_pipeline_steps)

# Numeric Feature Pipeline (min-max, box-cox)
numeric_pipeline_steps = []
numeric_pipeline_steps.append(('robust', RobustScaler()))
#numeric_pipeline_steps.append(('min-max', MinMaxScaler(feature_range=(1,2))))
#numeric_pipeline_steps.append(('box-cox', PowerTransformer(method='box-cox')))
numeric_pipeline = Pipeline(steps=numeric_pipeline_steps)

# Preprocessing Transformer
preprocessing_transformer_steps = []
preprocessing_transformer_steps.append(('cat', categorical_pipeline,␣
 ↪categorical_features))
preprocessing_transformer_steps.append(('num', numeric_pipeline,␣
 ↪numeric_features))
preprocessing_transformer=ColumnTransformer(transformers=preprocessing_transformer_steps)

# target pipeline (min-max, box-cox)
target_pipeline_steps = []
#target_pipeline_steps.append(('min-max', MinMaxScaler(feature_range=(1,2))))
target_pipeline_steps.append(('boxcox', PowerTransformer(method=('box-cox'))))
target_pipeline = Pipeline(steps=target_pipeline_steps)

# Display Transformer
from sklearn import set_config
set_config(display='diagram')
preprocessing_transformer
```

[ ]:
```
ColumnTransformer(transformers=[('cat',
                                 Pipeline(steps=[('onehot',
OneHotEncoder(handle_unknown='ignore'))]),
                                 Index(['season', 'holiday', 'workingday',
'weather', 'dt_year', 'dt_hour',
       'dt_day'],
      dtype='object')),
                                ('num',
```

```
                                    Pipeline(steps=[('robust', RobustScaler())]),
                                    Index(['temp', 'humidity'], dtype='object'))])
```

7 Spot-Check Algorithms

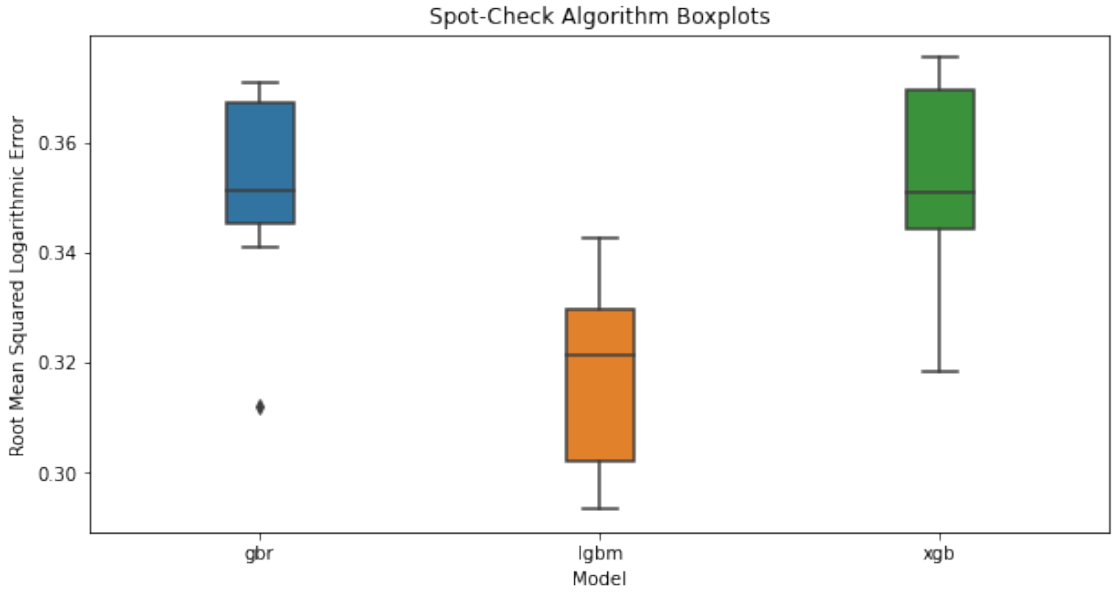
[ ]:
```
# Spot-Check Algorithms #####
# Store Results
results = pd.DataFrame()
scoring = 'neg_mean_squared_log_error'

# Models
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import Ridge, Lasso
models = []
models.append(('gbr', GradientBoostingRegressor(n_estimators=500)))
models.append(('lgbm', LGBMRegressor(n_estimators=500)))
models.append(('xgb', XGBRegressor(n_estimators=500)))

for name, model in models:
    # Pipeline
    model_pipeline_steps = []
    model_pipeline_steps.append(('transformer', preprocessing_transformer))
    model_pipeline_steps.append(('model', model))
    model_pipeline = Pipeline(steps=model_pipeline_steps)
    transformed_model = TransformedTargetRegressor(regressor=model_pipeline,␣
 ↪transformer=target_pipeline)
    # CV results
    cv_results = cross_val_score(transformed_model, X_train, y_train, cv=10,␣
 ↪scoring=scoring)
    # Changing to Root Mean Squared Logarithmic Error (RMSLE)
    cv_results = np.sqrt(abs(cv_results))
    # Append Results
    temp_df = pd.DataFrame({name: pd.Series(abs(cv_results))})
    results = pd.concat([results, temp_df], axis='columns')
    # Results Mean +/- std
    msg = "%s: %f +/- %f" % (name, cv_results.mean(), cv_results.std())
    # Print Results
    print(msg)
```

```
gbr: 0.351792 +/- 0.017140
lgbm: 0.316659 +/- 0.016526
xgb: 0.353495 +/- 0.016899
```

[ ]:
```
# Algorithm Comparison Boxplot
plt.figure(figsize=(10,5))
sns.boxplot(x='Model', y='Root Mean Squared Logarithmic Error',
            width=0.2,
            data=pd.melt(results, var_name='Model', value_name='Root Mean␣
 ↪Squared Logarithmic Error'))
plt.title('Spot-Check Algorithm Boxplots')
plt.show()
display(results)
```



	gbr	lgbm	xgb
0	0.359397	0.331410	0.368460
1	0.371029	0.324786	0.370689
2	0.370072	0.342650	0.375500
3	0.354296	0.332587	0.370039
4	0.345113	0.293301	0.341039
5	0.348439	0.320901	0.344912
6	0.340899	0.303349	0.344039
7	0.345664	0.301897	0.347822
8	0.311961	0.293631	0.318458
9	0.371048	0.322082	0.353997

8 Stacking Regressor

[ ]:
```
# import libraries
from sklearn.ensemble import StackingRegressor
# models
estimators = []
estimators.append(('lgbm', LGBMRegressor(n_estimators=500)))
estimators.append(('xgb', XGBRegressor(n_estimators=500)))
stack_reg = StackingRegressor(estimators=estimators,␣
 ↪final_estimator=LGBMRegressor(n_estimators=500))
```

8.1 Fit on Train

[ ]:
```
model = stack_reg
# Pipeline
model_pipeline_steps = []
model_pipeline_steps.append(('preprocessing', preprocessing_transformer))
model_pipeline_steps.append(('model', model))
model_pipeline = Pipeline(steps=model_pipeline_steps)
validation_model = TransformedTargetRegressor(regressor=model_pipeline,␣
 ↪transformer=target_pipeline)
validation_model.fit(X_train, y_train)
```

[ ]:
```
TransformedTargetRegressor(regressor=Pipeline(steps=[('preprocessing',
ColumnTransformer(transformers=[('cat',
       Pipeline(steps=[('onehot',
                        OneHotEncoder(handle_unknown='ignore'))]),
       Index(['season', 'holiday', 'workingday', 'weather', 'dt_year',
'dt_hour',
       'dt_day'],
      dtype='object')),
      ('num',
       Pipeline(steps=[('robust',
                        RobustScaler())]),
       Index(['temp', 'humidity'],…
                  monotone_constraints=None,
                  n_estimators=500,
                  n_jobs=None,
                  num_parallel_tree=None,
                  random_state=None,
                  reg_alpha=None,
                  reg_lambda=None,
                  scale_pos_weight=None,
                  subsample=None,
                  tree_method=None,
                  validate_parameters=None,
                  verbosity=None))],
```

```
final_estimator=LGBMRegressor(n_estimators=500))]),
                           transformer=Pipeline(steps=[('boxcox',
PowerTransformer(method='box-cox'))]))
```

[ ]:
```
# RMSLE on Validation Set
from sklearn.metrics import mean_squared_log_error
print('RMSLE on Validation Set: %f' % np.
 ↪sqrt(mean_squared_log_error(y_validate, validation_model.
 ↪predict(X_validate))))
```

```
RMSLE on Validation Set: 0.321041
```

9 Best Model on Entire Training Set

[ ]:
```
model = LGBMRegressor(n_estimators=500)
# Pipeline
model_pipeline_steps = []
model_pipeline_steps.append(('preprocessing', preprocessing_transformer))
model_pipeline_steps.append(('model', model))
model_pipeline = Pipeline(steps=model_pipeline_steps)
final_model = TransformedTargetRegressor(regressor=model_pipeline,␣
 ↪transformer=target_pipeline)
final_model.fit(X, y)
```

[ ]:
```
TransformedTargetRegressor(regressor=Pipeline(steps=[('preprocessing',
ColumnTransformer(transformers=[('cat',
       Pipeline(steps=[('onehot',
                        OneHotEncoder(handle_unknown='ignore'))]),
       Index(['season', 'holiday', 'workingday', 'weather', 'dt_year',
'dt_hour',
       'dt_day'],
      dtype='object')),
      ('num',
       Pipeline(steps=[('robust',
                        RobustScaler())]),
       Index(['temp', 'humidity'], dtype='object'))])),
                                                    ('model',
LGBMRegressor(n_estimators=500))]),
                           transformer=Pipeline(steps=[('boxcox',
PowerTransformer(method='box-cox'))]))
```

9.1 Predictions

[ ]:
```
# Predictions
predictions = final_model.predict(test)

# Submission
```

```
from datetime import datetime
submission_model = 'LGBM'
submission = pd.DataFrame({'datetime':sample_submission['datetime'], 'count':␣
 ↪predictions})
submission_file_name = 'submissions/' + submission_model + ' on ' + datetime.
 ↪now().strftime("%Y %b %d at %H.%M.%S") + '.csv'
submission.to_csv(submission_file_name, index=False)
submission.head(5)
```

[ ]:

	datetime	count
0	2011-01-20 00:00:00	12.686467
1	2011-01-20 01:00:00	3.988990
2	2011-01-20 02:00:00	3.600739
3	2011-01-20 03:00:00	2.583370
4	2011-01-20 04:00:00	1.769149